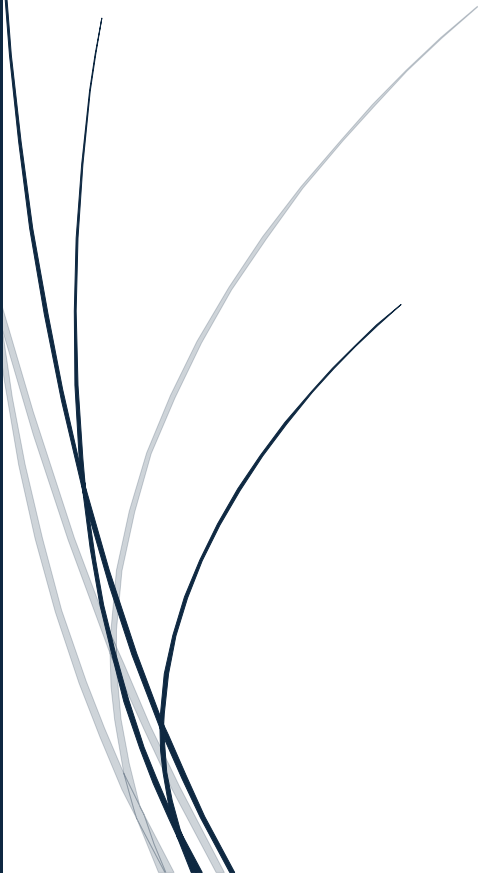




5/11/2026

REPIF

Alff Lee



Lee Alff
EMILE METZ

Table of Contents

Userstory A1	4
PVC Holder	4
Userstory A2	0
PCB Assembly	0
Introduction	0
Fabrication Steps	0
Soldering	0
Attaching to PVC Holder	2
Userstory L3	4
Power supply cable	4
Introduction	4
Fabrication steps	4
Soldering it together	6
Userstory A4	9
Introduction	9
Steps	9
Verification of Sensors	10
Component List	11
Userstory A5	11
Introduction	11
Steps	11
Verification of Sensors	12
Component List	13
Userstory A6	13
Introduction	13
Steps	13
Showing the completed casing and proof, it works	17
Component List	17
Userstory A7	18
Introduction	18
Steps	18
LAB WORK	22
Userstory L1	22
Install Raspberry Pi OS	22
2.3 Install Docker	23

2.4 File Structure and Deployment	23
Create Docker Container (php:8.4-apache)	24
Autostart Configuration	24
Screenshots / Evidence	24
Userstory L2	25
Conceptual model	25
Logical model	25
Physical model	26
Userstory L3	27
Userstory L4	2
Setting up the Machine	2
Install PHP & Apache modules	2
Configure web directory permissions	3
Install phpMyAdmin	3
Setup Database	4
Create database and import schema	4
Install PHP extensions	4
Create database user and grant privileges	5
SSL Certificate Setup	5
Install OpenSSL and generate certificate	5
Configure Apache for HTTPS	6
Userstory L5	7
stationInput.php	7
Config.toml	7
Proof	7
Userstory L5	8
1. Prerequisites	8
2. Install MySQL Client	8
3. Create Backup Directory & User	8
4. Configure MariaDB on the Database Server	8
4.1 Create a MySQL Backup User	8
4.2 Allow External Connections	9
5. Create the Backup Shell Script	9
6. Schedule Daily Backup at 8:00 AM	10
7. Configure FTP Access	10
7.1 Install vsftpd	10
7.2 Create FTP User	10

7.3 Configure vsftpd	10
8. Verify the Backup	11

Userstory A1

PVC Holder

On the base plate there are 14 holes to be drilled.

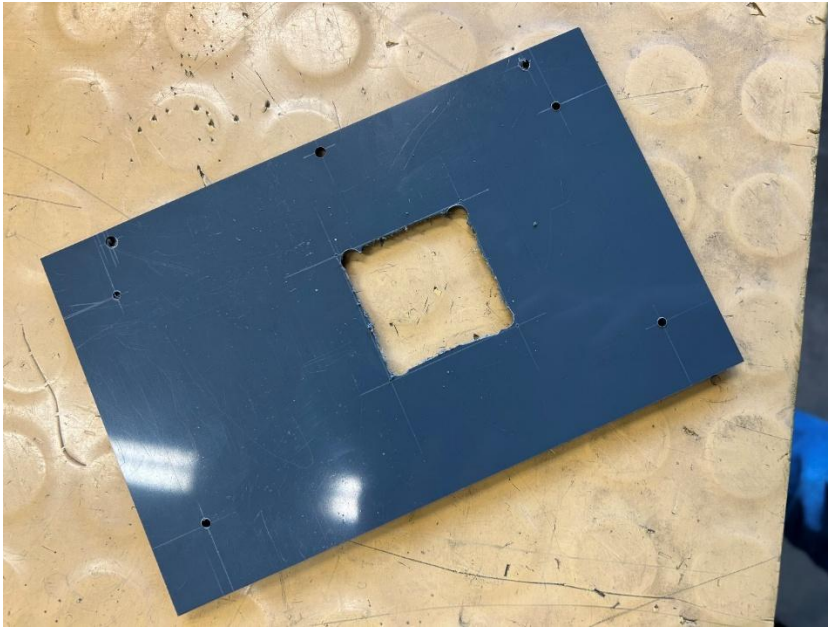
On the right side there are 4 2,5 millimeter holes that will later have the raspberry pi installed 4 more 2,5 millimeter holes and one 4mm hole in the middle. There are also three 3 millimeter holes on the side to attach the display plate to.



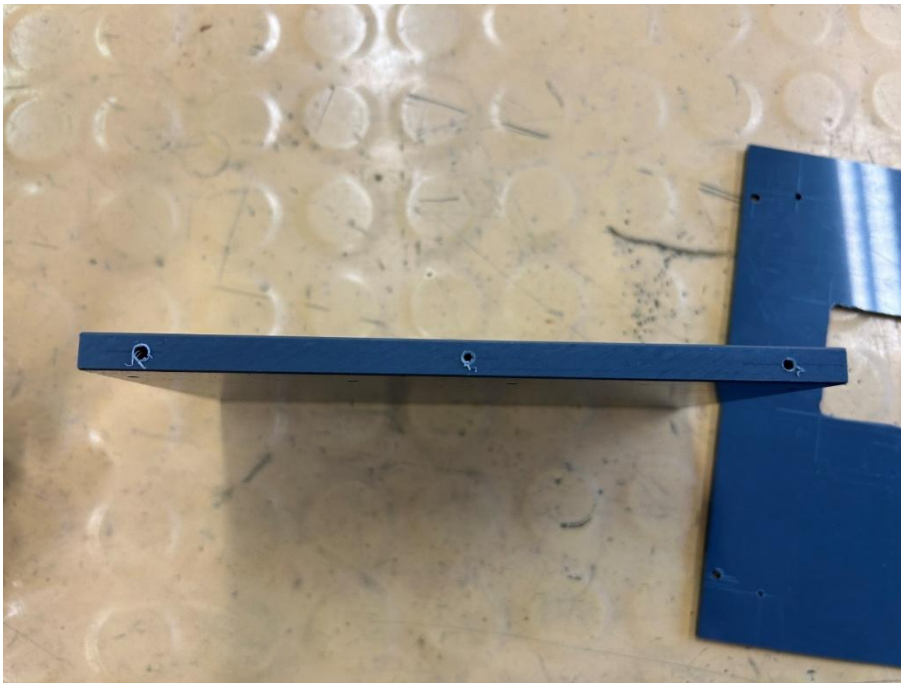
On the screen holder plate there is a square cutout for cabling and 4 2,5 millimeter holes to have the screen installed and three 3 mm holes to attach the base plate to.



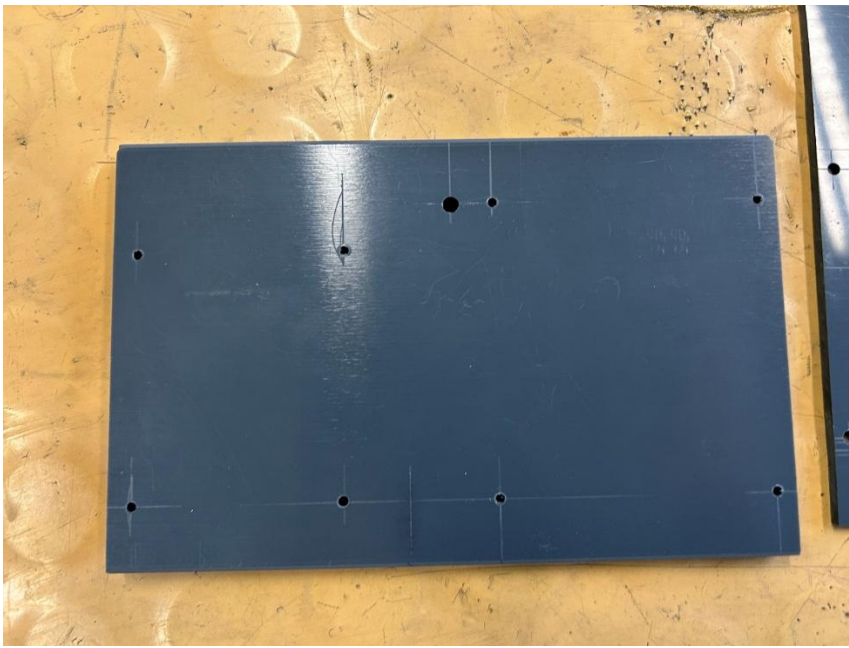
After drilling the holes, I cut out the middle part for the display plate



On the display plate for the holes on the side I needed to drill with a 2,5 mm first and then with a 3mm special drill to make screwholes



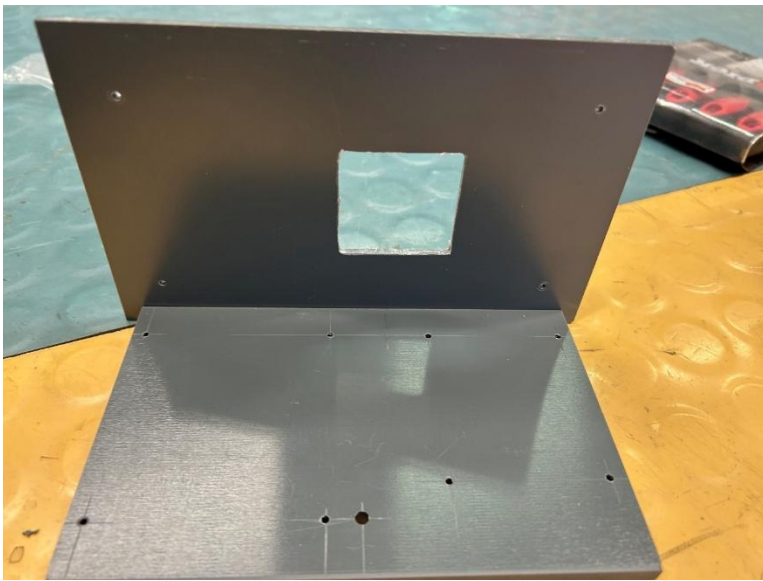
Here is the bas plate with the holes drilled



I need to debur the screw holes, so they fit smoothly



Once finished with the deburring I can attach both plates to each other



Now I can attach the raspberry pi



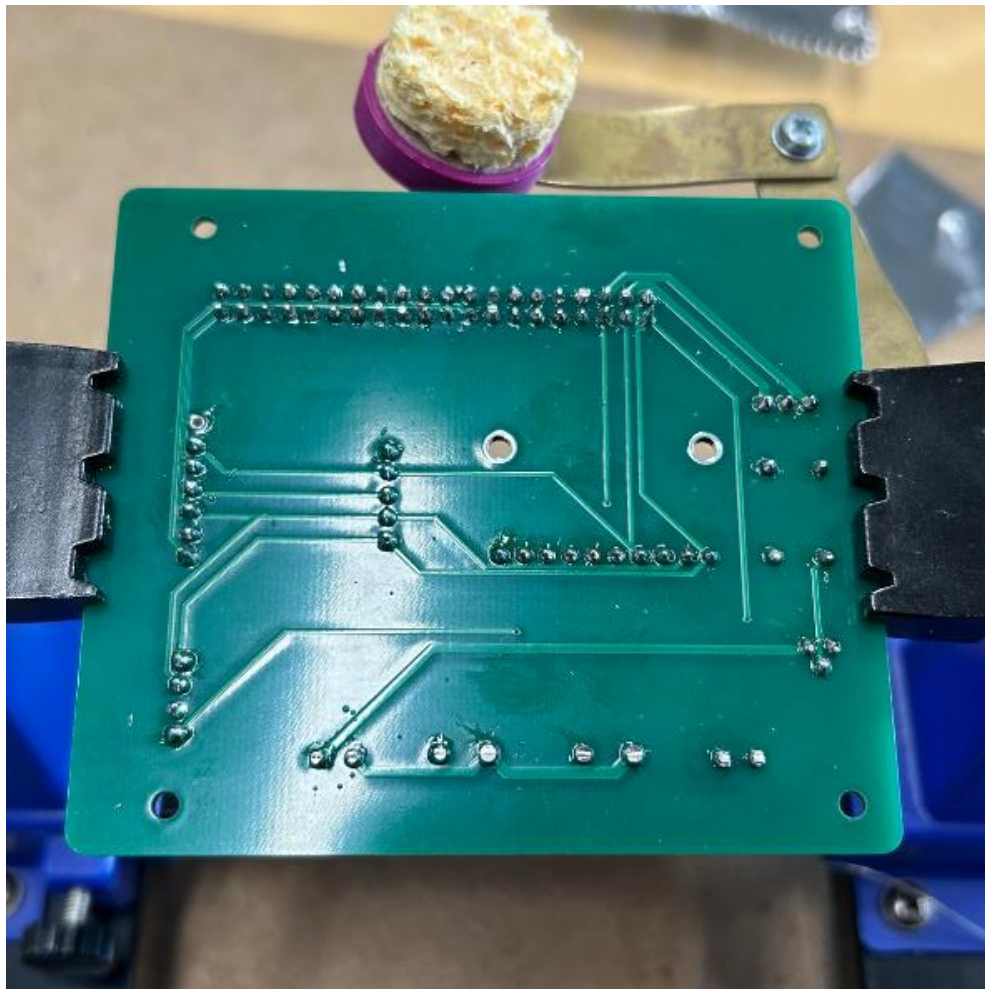
And afterwards I can attach the screen



Finished it should look like this



Here is an image of the unsoldered circuit board



Here is the fully soldered circuit board

You have to be very sure that everything is soldered on straight. Something not being properly aligned is usually not a fatal mistake that will interrupt much with the functionality (unless it is extremely misaligned) it is still a mistake, which you can usually fix with a solder sucker.

Which you use by heating up the solder and then you vacuum the liquid solder.

Attaching to PVC Holder

In this step we simply need to screw the board to the rest of the project with the project.

For this I usually start placing down the spacers first and place the board on top of it and screw it one side side and the the opposite site. By then the two other spaces usually misalign but since the board is now mostly stationary I can just realign it and tighten it without any problems.

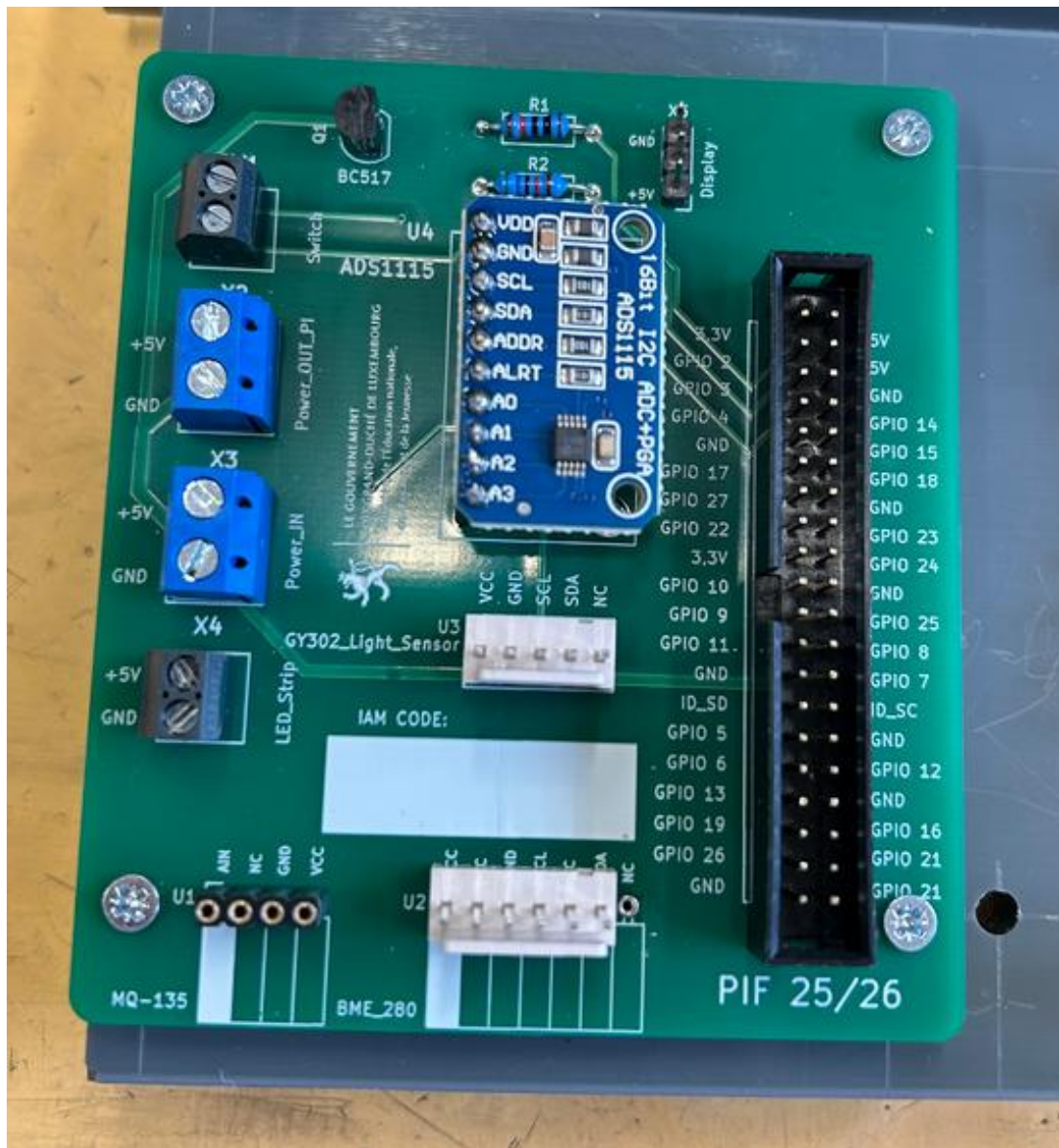
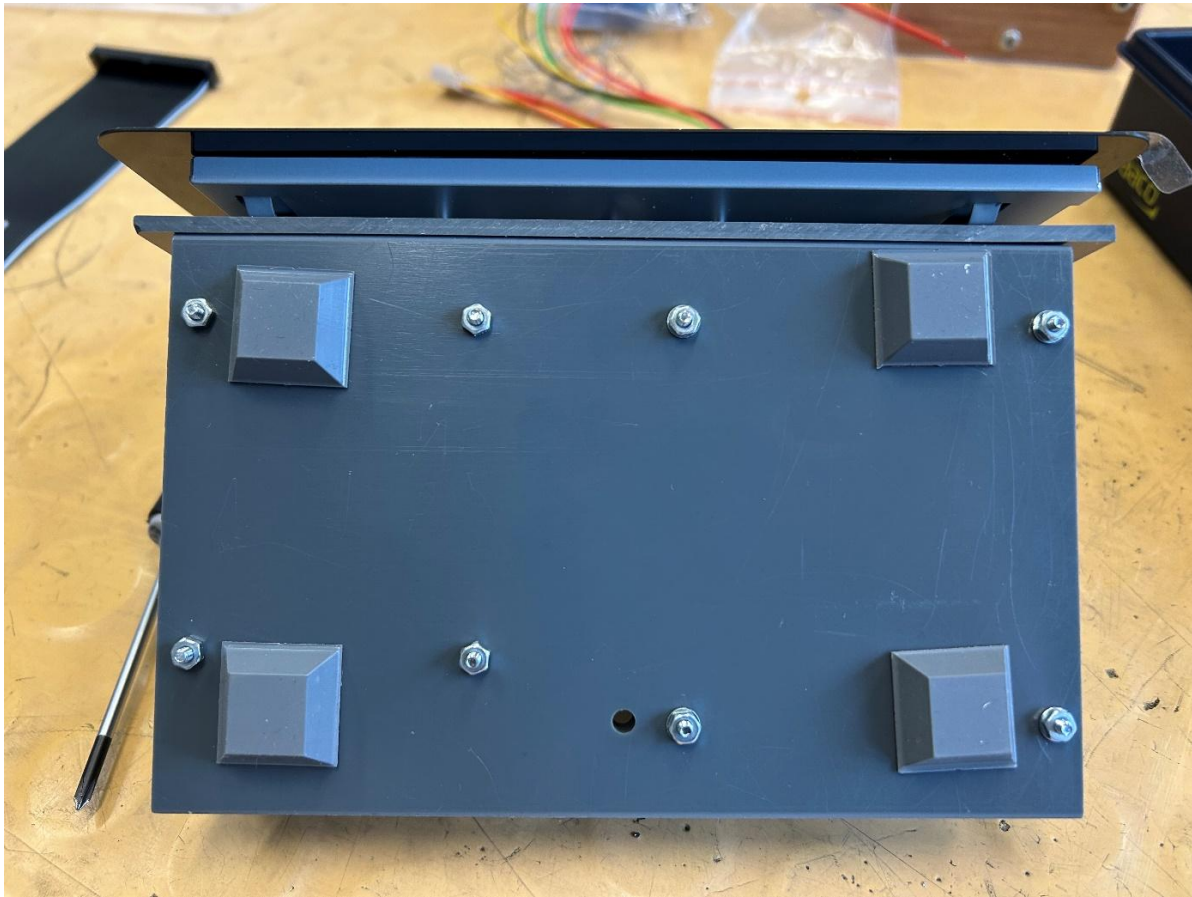


Image of the board fully mounted

Then I turn the project around to attach the washers and nuts to the screws to properly secure the plate.



Underside of the board with the nuts and washer attached

And now the PCB is mostly assembled

Userstory L3

Power supply cable

Introduction

The goal of this sprint was to learn how to combine two separate cables into one function cable for a power supply.

Fabrication steps

Preparing the outer layers

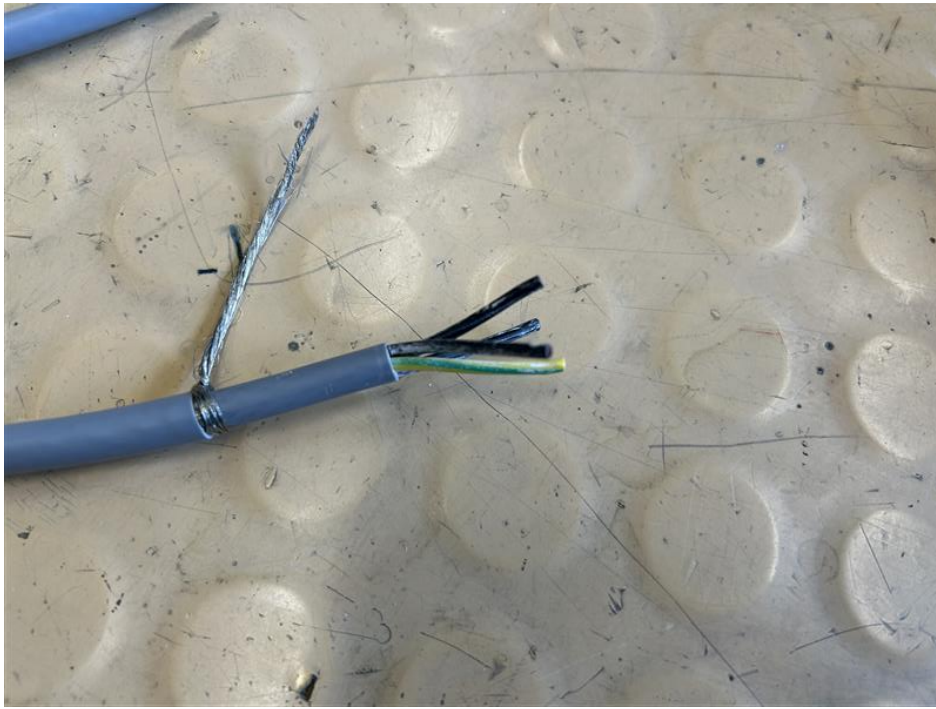


First I have to remove the outer insulation. I have to consider an appropriate amount of length to remove. Once marked I will use the wirestripper to remove the outer layer.

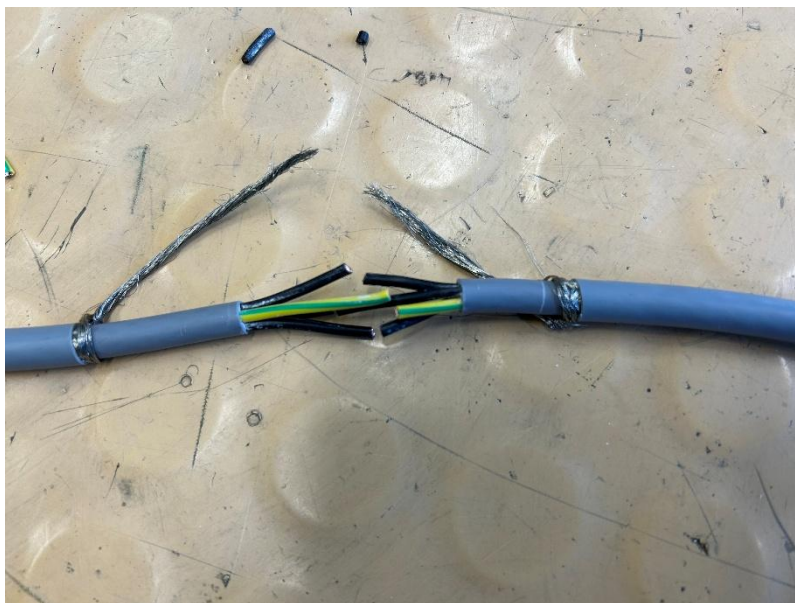
After that you need to twist the aluminium sheath so that it does not interfere with the rest of the work and so you can solder it later.



Now I need to remove the third layer of insulation to expose the 4 wires.



Preparing the wires



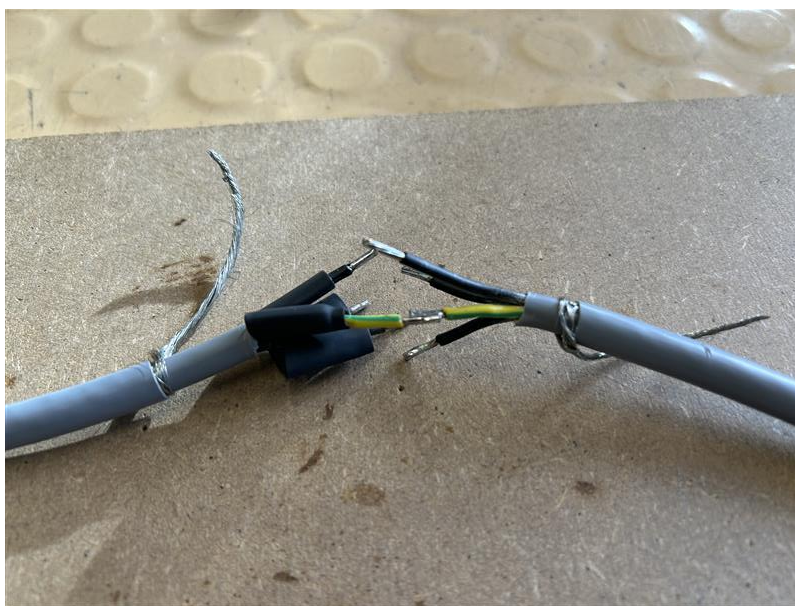
Then I will cut the cables at different positions. This is called staging. It is done primarily for safer work and also makes it easier to work on it.

After which I need to remove the insulation for the wires.

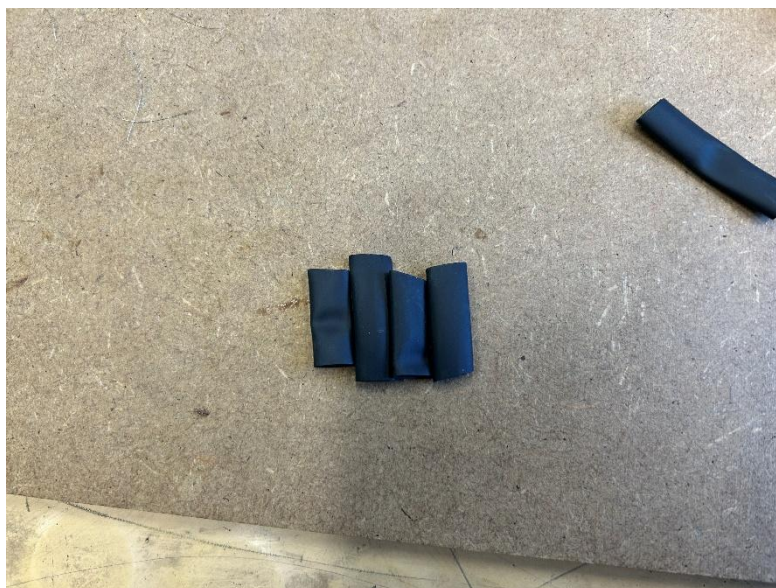


Soldering it together

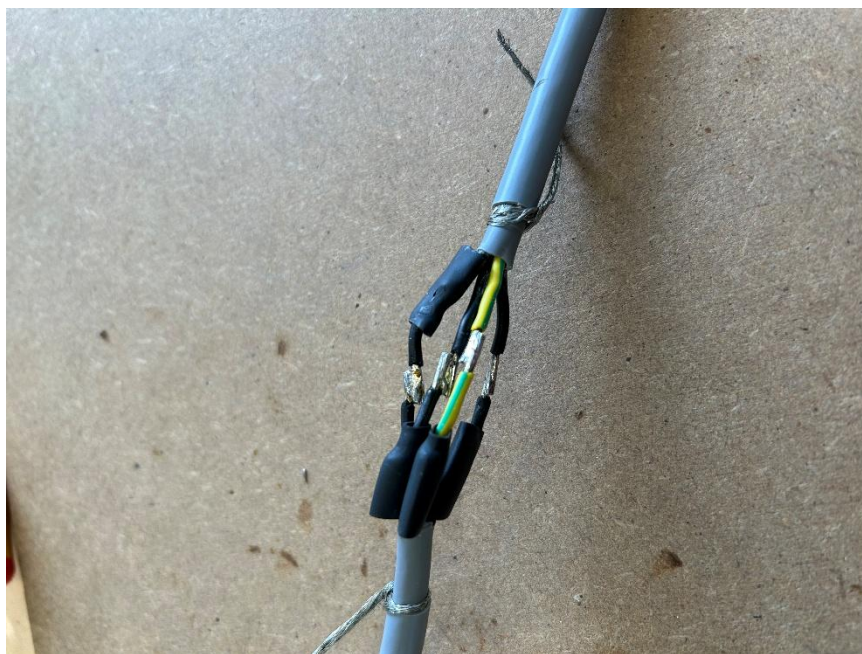
Then I put solder on the exposed ends of the wires



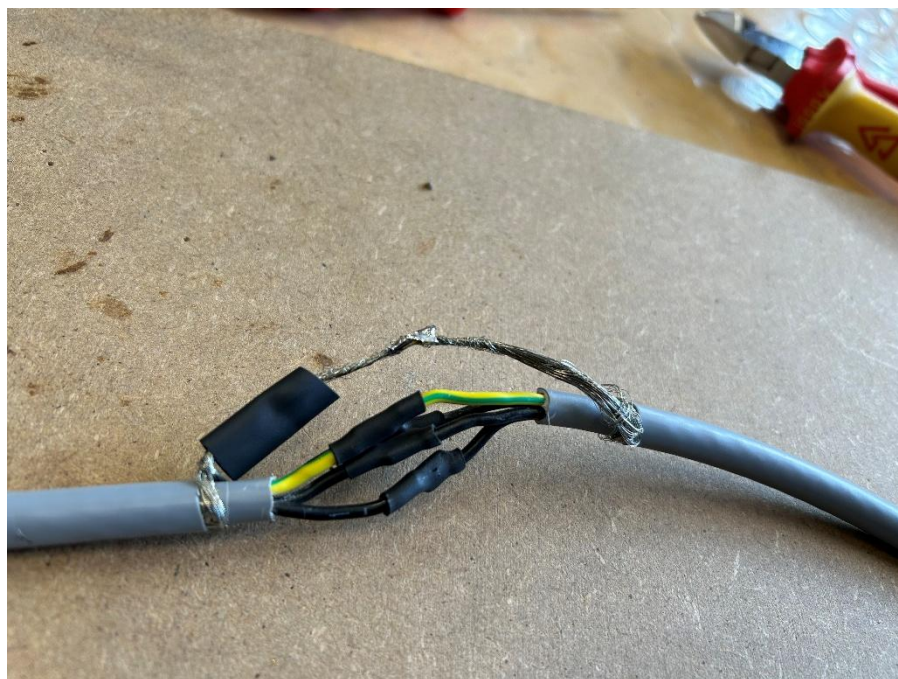
After that I will also prepare the new insulation off a bigger strip.



Now I will fuse all the cable ends by heating the wires so the solder fuses with each other's ends and create a connection.



I also need to solder the aluminium sheath to each other



After this I just need to heatgun the insulation so that it tightens and properly protects the exposed wire

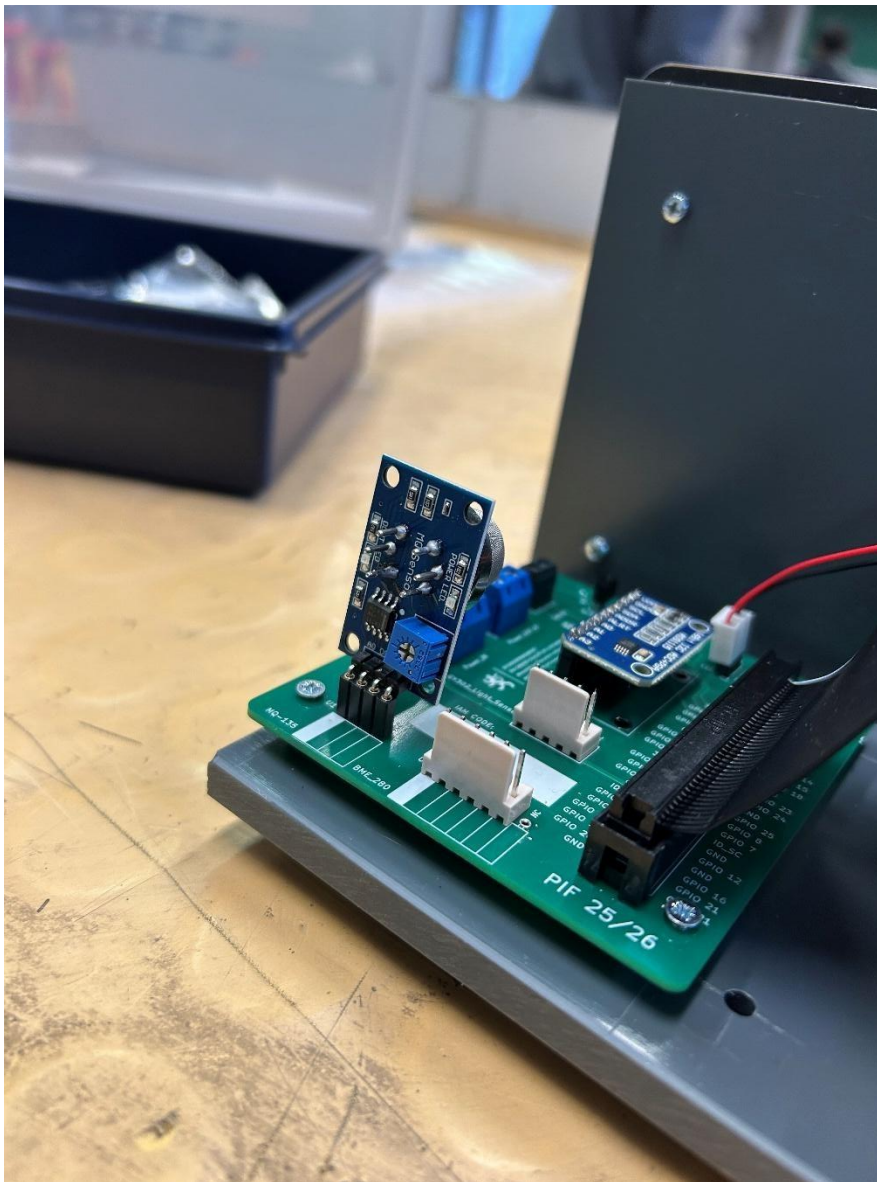


Userstory A4

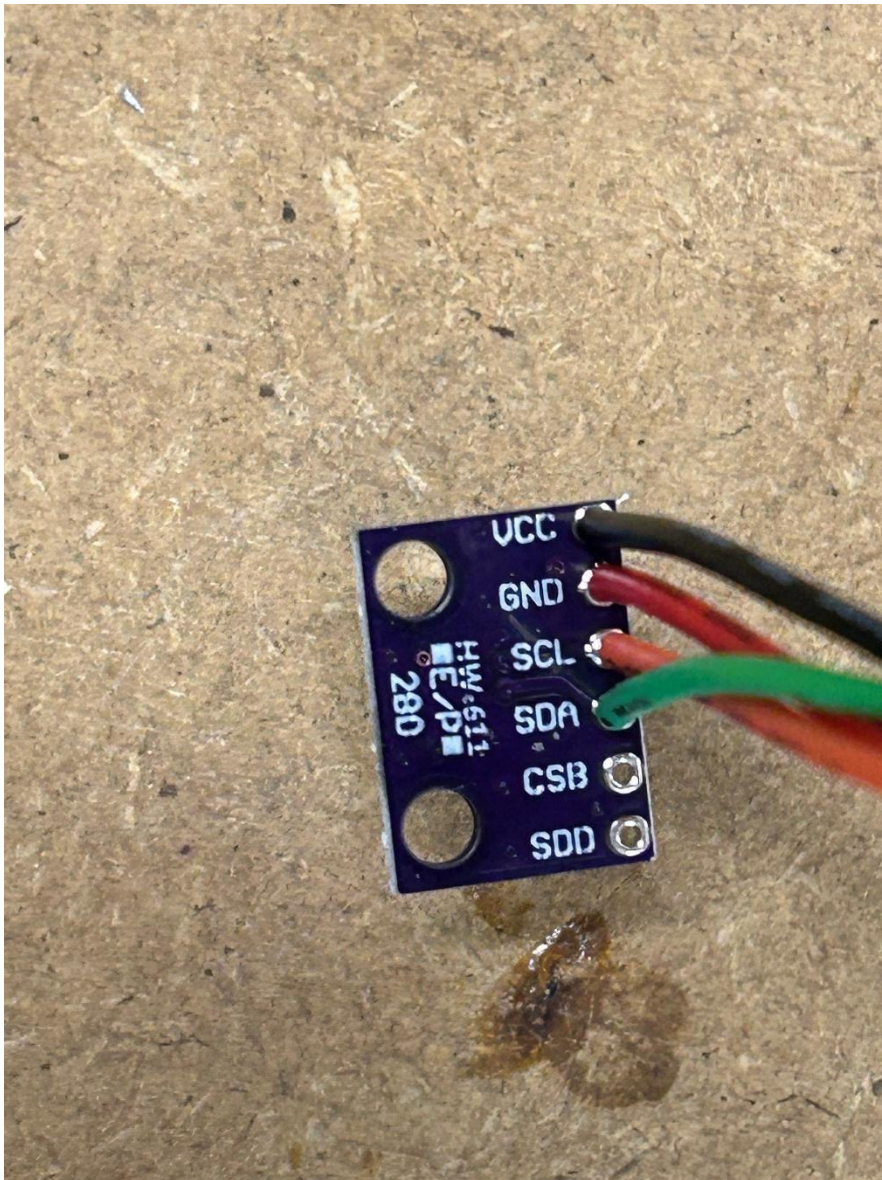
Introduction

For this sprint we had to install the sensors and test their functionality.

Steps



For the Gas Sensor I just need to plug it in so there are no big problems there.



This one was quite tricky to solder. It is quite hard to keep the component still since you can't really grab it without potentially burning your hand. I recommend getting a colleague to help or fix the part in place. What also immensely helped was me fixing the cables in place.

Verification of Sensors

To test the sensors, I must use the provided test files which activate the sensors to take a singular scan. To use them properly I first must go into the code and switch out the i2c address. In this case the gas sensor has address 46 and the light sensor 28.

After that I can activate them with the command “python3 test_1750.py”

Component List

Component List
BH 1750 Sensor
BME 280 Sensor

Userstory A5

Introduction

For this sprint we had to install a light sensor and a temperature sensor.

Steps

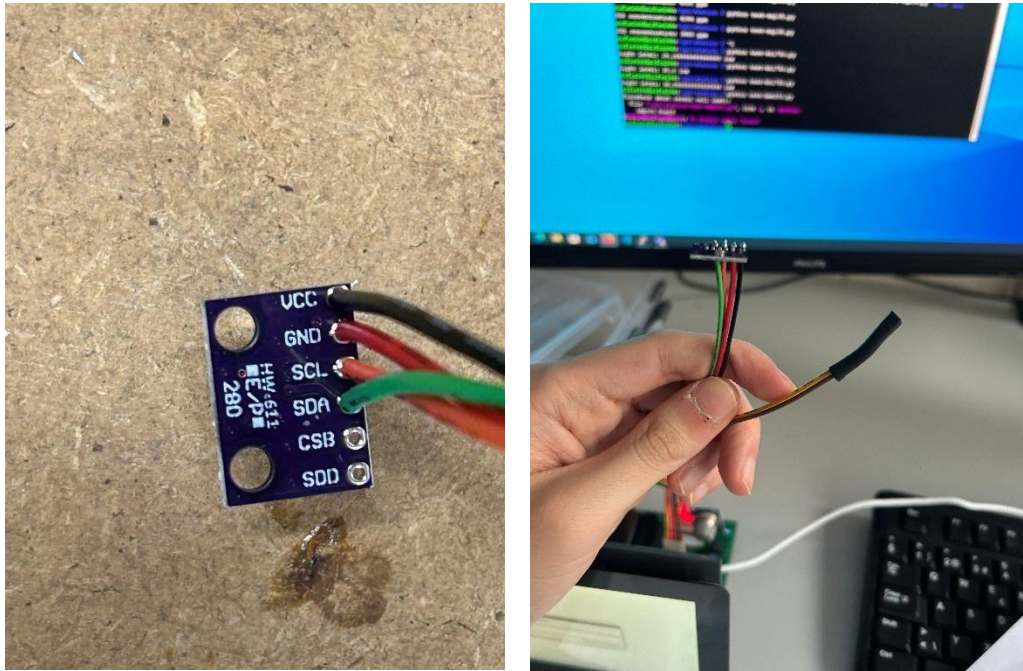
Light Sensor



Because these sensors are small components, they can be finicky to work with and usually either require some help from a person or something that fixes them in place.

Outside of that this sensor was working right away.

Temperature Sensor



This one had the same problems as the last sensors with the addition of needing to isolate two redundant cables so that they cannot touch any components.

However where the true problem with this sensor lies is that the software with this sensor was not actually compatible right out of the gate and we had to change a chipset id so that it could actually work.

Verification of Sensors

To test the sensors, I must use the provided test files which activate the sensors to take a singular scan. To use them properly I first must go into the code and switch out the i2c address. In this case the temperature sensor has address 74 and the light sensor 28.



MicrosoftTeams-video.mp4

Temperature Sensor Proof



MicrosoftTeams-video(1).mp4

Light Sensor Proof

Component List

Component List
Button
Wooden Plates

Userstory A6

Introduction

In this sprint we created the wooden casing for our weather station with a corresponding button to activate the LED.

Steps

Wooden Casing



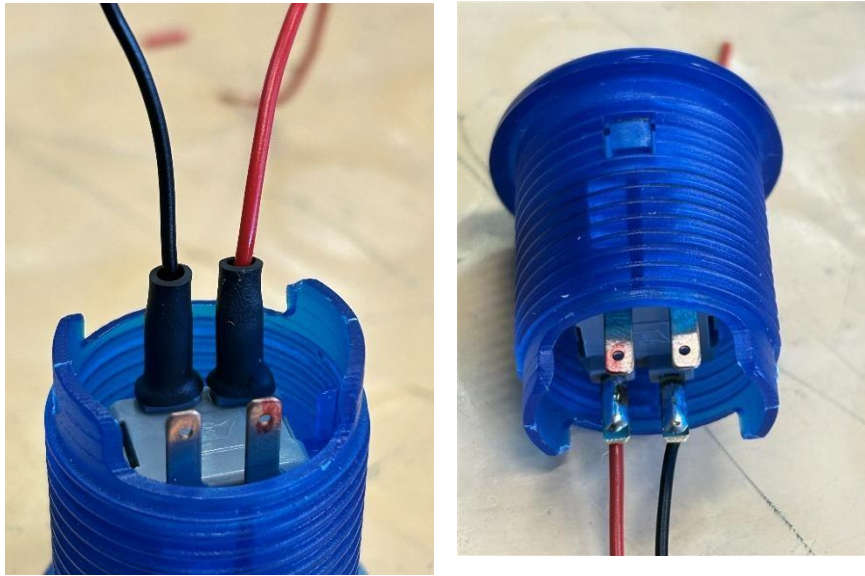
The beginning was quite easy as you only had to insert two joints and then attach the walls to the plate. There were no problems here.

Sensor Screw Holes

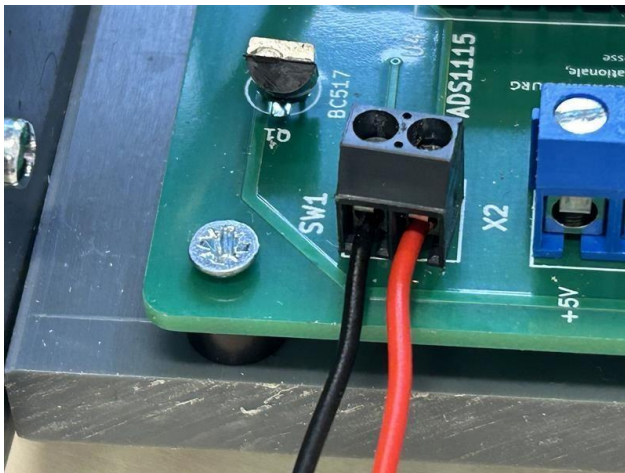


And then because we attach the sensors to the backplate we had to drill four holes into the plate which will later hold the sensors.

Preparation of the Button

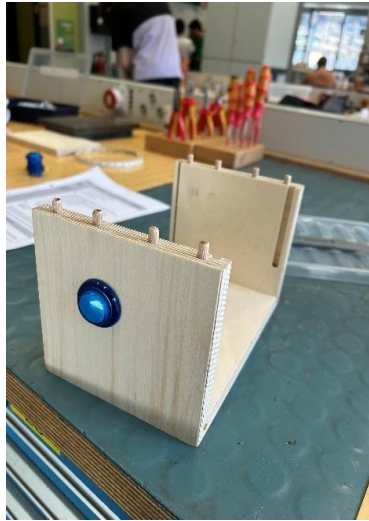


This was a lot harder since the button tends to move around while one is trying to solder the cables to it but just using a friend or something to hold it in place will fix that problem. Once I was done soldering I also added isolation for extra security.



After this I can connect the two cables to the PVC plate. This had no problems.

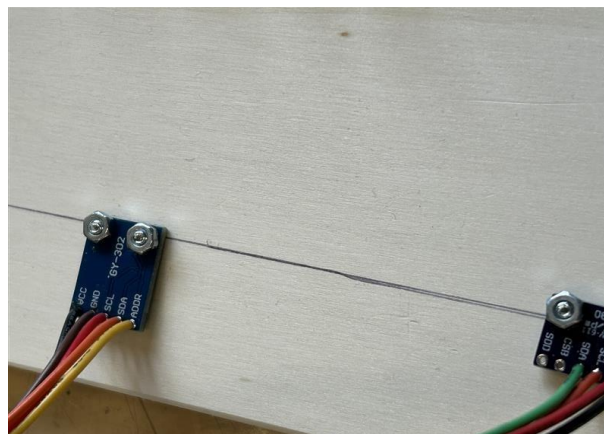
Assembling the case



Once I connected the cables I only had to put everything together like Lego.



And I had to remove the feet which were previously attached to the plastic plate that held the raspberry pi and now put it on the new casing.



And to finish it I screwed the sensors to the backplate.

Showing the completed casing and proof, it works



MicrosoftTeams-video.mp4

Video Proof that everything is still updating.

Component List

Component List
LED Strip

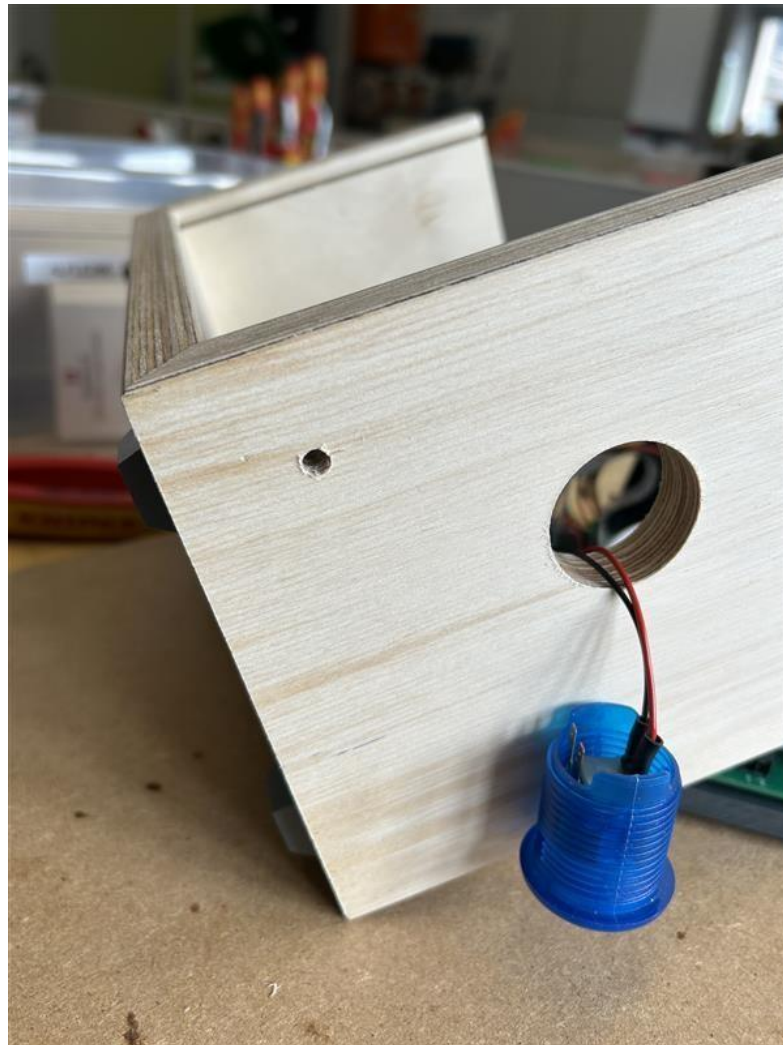
Userstory A7

Introduction

In this sprint we attach the LED strip to our project.

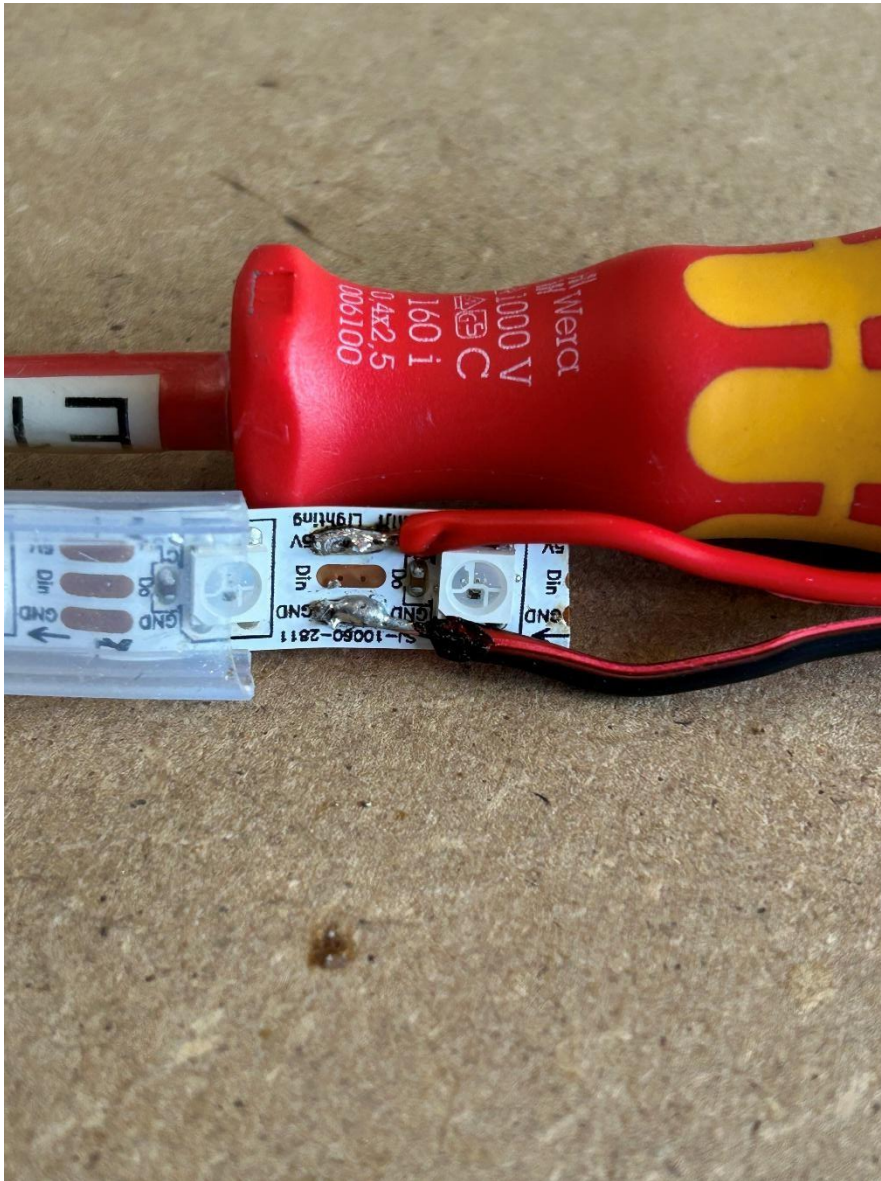
Steps

Wooden Casing Hole



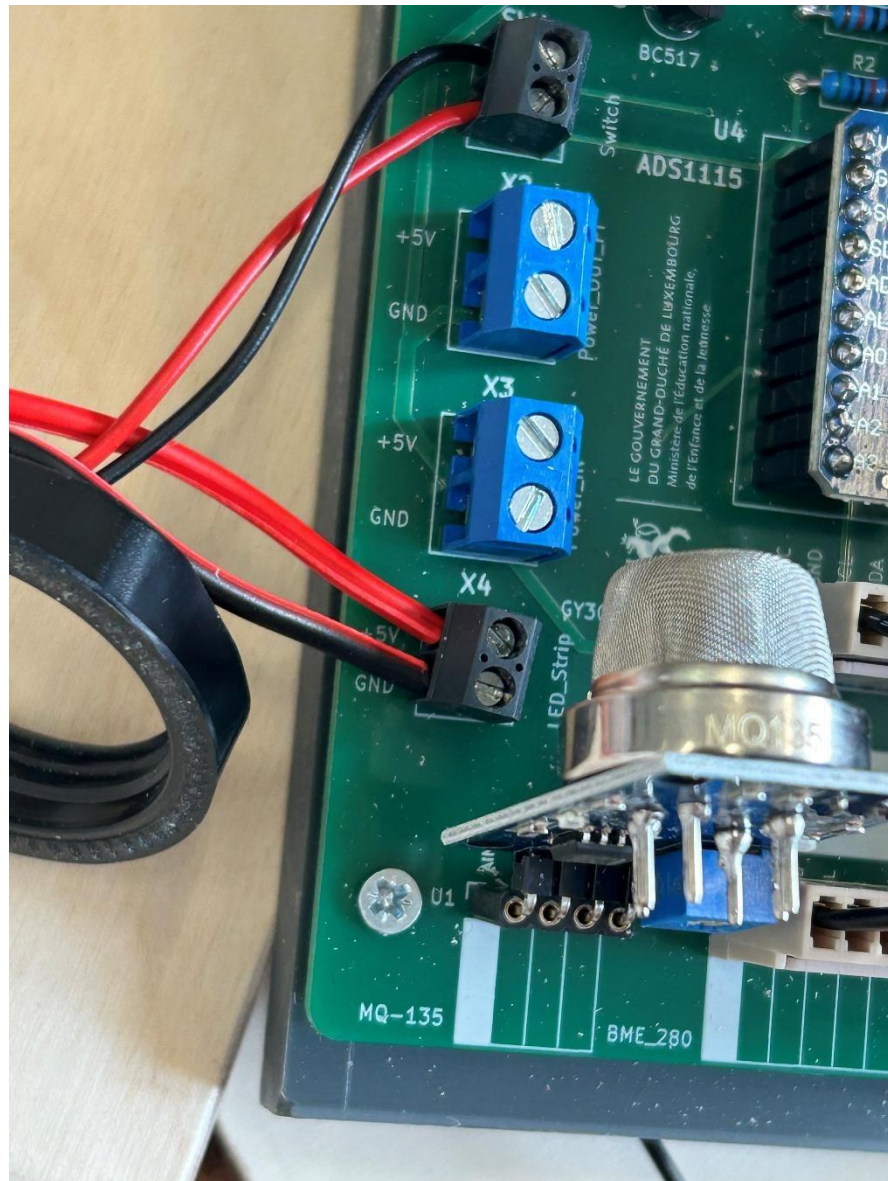
We had to drill a hole into the casing so we could connect the cable of the LED strip to the circuit board.

Connecting the LED strip



Now we had to solder the cables to the LED strips which proved quite challenging. It was quite hard to have the solder connect to the strip itself.

Connecting it to the Board



And now we just have to connect the cables to the board itself.

Attaching the strip



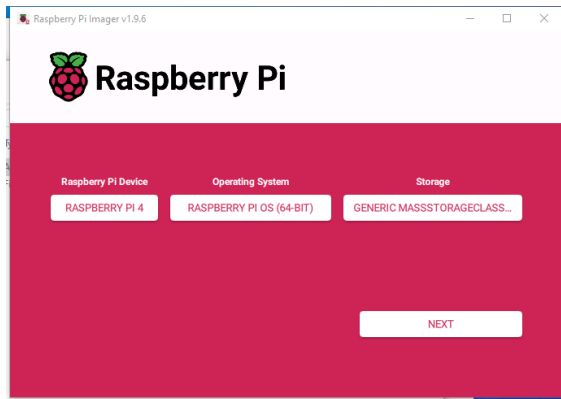
Now I only need to glue the strip to the case.

LAB WORK

Userstory L1

Install Raspberry Pi OS

- Install **Raspberry Pi OS (64-bit)** onto the microSD card.



- **2.2 Configure Basic System Settings**

☒ Set hostname: .local

☒ Set username and password

Username:

Password:

- **Set hostname and enable SSH**

☒ Enable SSH

☐ Use password authentication

☐ Allow public-key authentication only

Set authorized_keys for 'alfle266':

DELETE KEY

RUN SSH-KEYGEN ADD SSH KEY

- **Configure static IP.**

```
sudo apt install dhcpcd5
Get:1 http://archive.raspbian.org/raspbian/ buster/main amd64 dhcpcd5 amd64 1:6.11.0-1 [10.5 kB]
$ sudo nano /etc/dhcpcd.conf
```

At the end of the file add this. (Replace wlan0 with eth0)

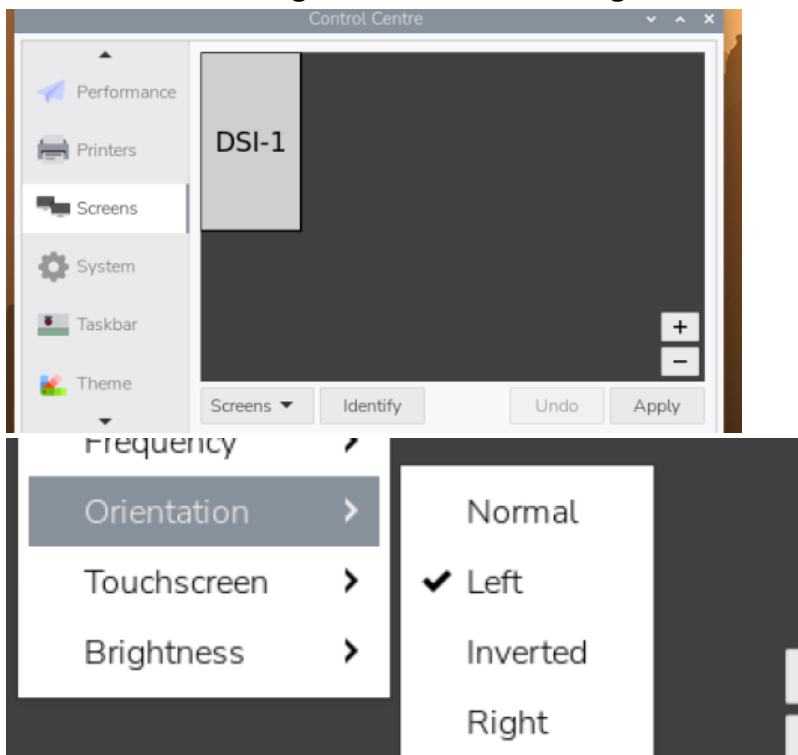
```

interface wlan0
static ip_address=192.168.4.215/24
static routers=192.168.4.1
static domain_name_servers= 192.168.4.1
$ sudo systemctl start dhcpcd
$ sudo systemctl enable dhcpcd
of dhcpcd.service with SysV service script
/systemd/systemd-sysv-install enable dhcpcd

```

- **Set screen orientation (landscape).**

In the control centre go to screens and change orientation



2.3 Install Docker

Run installation script: `$ curl -fsSL https://get.docker.com -o install-docker.sh`

2.4 File Structure and Deployment

Create folders:

```
alfle266@alfle266:/opt $ sudo mkdir station
```

```
alfle266@alfle266:/var $ sudo mkdir station
```

Copy supplied files:

Website files → /opt/station/www

```
alfle266@alfle266:~/Desktop/Ressourcen/Station-Webseite $ sudo mv * /opt/station
```


Firmware and scripts → /opt/station/www

```
alfle266@alfle266:~/Desktop/Ressourcen/Firmware und Scripts $ sudo mv * /opt/station
```

Adjust configuration file:

```
alfle266:~/config/labwc $ sudo nano /opt/station/config.toml
```

Change the IP destination

```
#ip = 192.168.6.215
```

Create Docker Container (php:8.4-apache)

```
alfle266@alfle266:~ $ docker run -d --name php --restart unless-stopped -p 8080:80 -v /tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache
```

Autostart Configuration

Create the autostart file

```
alfle266@alfle266:/tmp/station $ cd ~/config/labwc
alfle266:~/config/labwc $ sudo nano autostart
```

```
GNU nano 8.4 autostart *
mkdir -p /tmp/station
python3 /opt/station/measure.py > /tmp/station/result.csv &
sudo systemctl start docker

docker start php

sleep 5

chromium http://localhost:8080 --kiosk --noerrdialogs --disable-infobars --no-f
restore-last-session=false \
--incognito \
```

Screenshots / Evidence

When everything is done correctly this should appear on startup in Kiosk mode

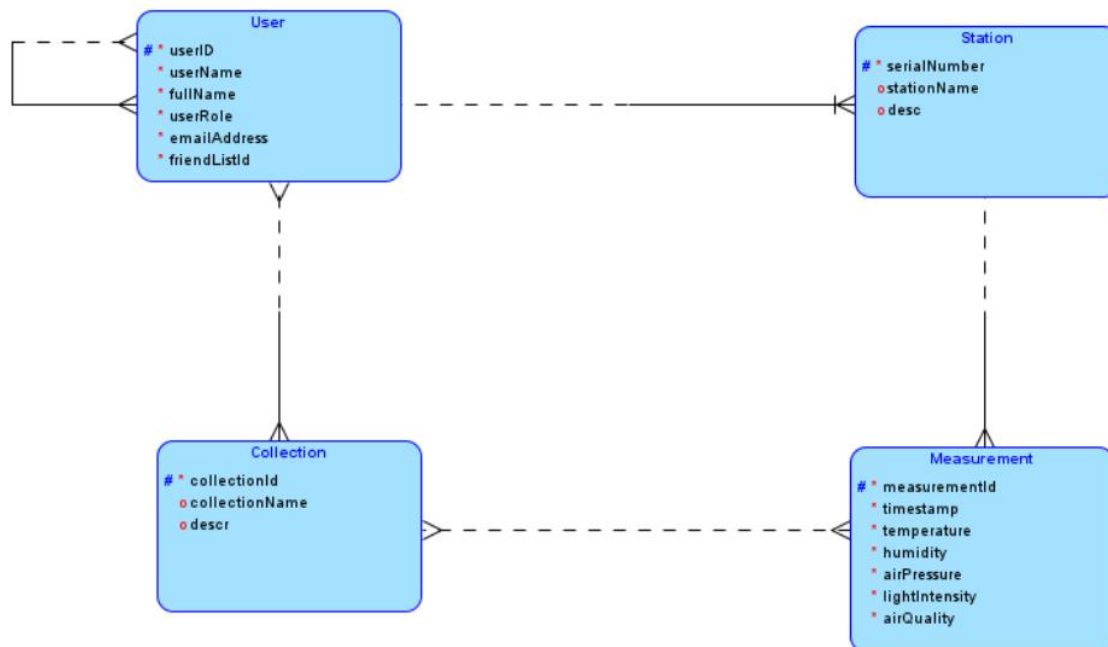
Time	2025-11-20 15:54:17
Temperature	20.00 °C
Humidity	50.00 %
Pressure	1,000.00 hPa
Light	0.00 lux
Gas	500.00 ppm

Userstory L2

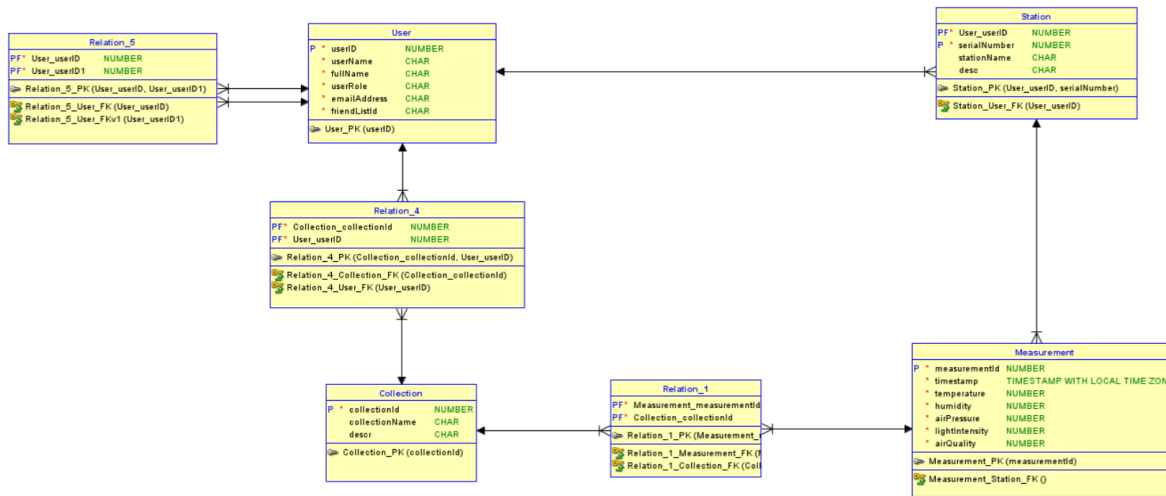
Conceptual model



Logical model



Physical model



Userstory L3

A User Story	L3 - Website	to	KW 05
Informal description	As a user or administrator of a station		
	... I want a functional and user-friendly website		
	... so that I can manage and share stations and measured values.		
Requirements according to the work order	Required • <u>Documentation:</u> ○ Annotated source code of the website ○ Planning this user story Without login: • Login with username and password is possible. • Registration/creation of a new user account is possible. Logged-in users: • Logout is possible. • A simple welcome page is displayed. • Your own stations can be listed. The name and description of the stations can be changed. • Stations not yet assigned to a user can be registered by serial number. • Your own user account (username, password, email, first and last name) can be edited. • Measurement data from your own stations can be viewed and filtered by date/time. • Users can be added as friends. Friends can be displayed and friendships can be ended. • Measurement data can be added to collections. Users can only select from their own stations. A collection contains measurement data from a specific station from a start date/time to an end date/time. Your own collections can be renamed and deleted. • Collections can be shared with friends. Sharing can be undone. • Collections shared with the user can be viewed. Admins (additional):		

	• All user accounts can be edited and deleted. Admin rights can be assigned to users. New user accounts can be created. • All stations can be edited and deleted. New stations can be created. • Measurement data from all stations can be viewed, filtered by date/time, and deleted. • Measurement data from all stations can be added to collections. All collections can be renamed and deleted. Admins can only share collections they have created themselves. Should • The welcome page contains a dashboard that displays current measurement data from your own stations. • The website has an appealing and user-friendly design. • User entries are validated. • Users cannot be added as friends directly, but via a friend request system that requires confirmation of the request. • When a friendship is ended, all collections shared between the two users are automatically unshared. Could • Registration of stations via QR code. • Measurement data is displayed using diagrams. • The website automatically adapts to different screen sizes (responsive). • Different themes (e.g., light/dark mode). The selected theme is retained even after logging out. • The website is available in different languages, and the selected language remains unchanged after logging out. • Users can be notified of various events (friend requests, sharing of measurement data, errors, etc.) via email and/or notifications. • Users can chat with each other on the website. • Users can send invite links to invite new users to the platform.
Tasks according to the work assignment	
Planning	
• Analysis of the user story and differentiation between guest, user, and admin roles • Rough planning of database structure (users, stations, measurements, friendships,	

collections, sharing)

- Definition of basic page flow and navigation
-

User management

- Implementation of user registration with validation
 - Login and logout functionality
 - Welcome page for logged-in users
 - Editing of user account data (username, password, email, first and last name)
-

Station and measurement management

- Display of stations assigned to the logged-in user
 - Editing of station name and description
 - Registration of unassigned stations via serial number
 - Display of measurement data for own stations
 - Filtering of measurement data by date and time
-

Friend system

- Implementation of a friend request system
 - Accepting and declining friend requests
 - Display of friend list
 - Ending friendships
-

Collections and sharing

- Creation of collections based on station and time range
 - Display, renaming, and deletion of own collections
 - Sharing collections with friends and revoking access
 - Display of collections shared with the user
-

Admin functionality

- Editing and deleting user accounts
 - Assigning admin rights and creating new users
 - Creating, editing, and deleting stations
 - Viewing, filtering, and deleting measurement data
 - Managing collections (rename, delete)
-

UI and documentation

- Implementation of a simple, user-friendly interface
- Notation of source code and documentation of implementation

Userstory L4

Setting up the Machine

Install Apache, PHP, and configure web directory permissions before proceeding to the database.

Install PHP & Apache modules

```
alfle266@alfle266:~$ sudo apt install php-common libapache2-mod-php php-cli_
```

Install core PHP packages and the Apache PHP module.

```
alfle266@alfle266:~$ sudo apt install mariadb-server -y_
```

Install MariaDB with the -y flag to auto-confirm all prompts.

```
alfle266@alfle266:~$ sudo mysql_secure_installation_
```

Run the security hardening wizard to set the root password and lock down defaults.

```
Remove anonymous users? [Y/n] y
... Success!

Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] y
... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access. This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] y
- Dropping test database...
... Success!
- Removing privileges on test database...
... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] y
... Success!

Cleaning up...

All done! If you've completed all of the above steps, your MariaDB
installation should now be secure.
```

Answer Y to all prompts: remove anonymous users, disallow remote root login, drop the test database, and reload privileges.

Configure web directory permissions

```
$ sudo chown -R www-data:www-data /var/www/html
$
```

Transfer ownership of the web root to the www-data user so Apache can serve files correctly.

```
$ sudo find /var/www/html/ -type d -exec chmod 755 {} \;
$ sudo find /var/www/html/ -type f -exec chmod 644 {} \;
```

Set directories to 755 and files to 644 — standard secure permissions for web content.

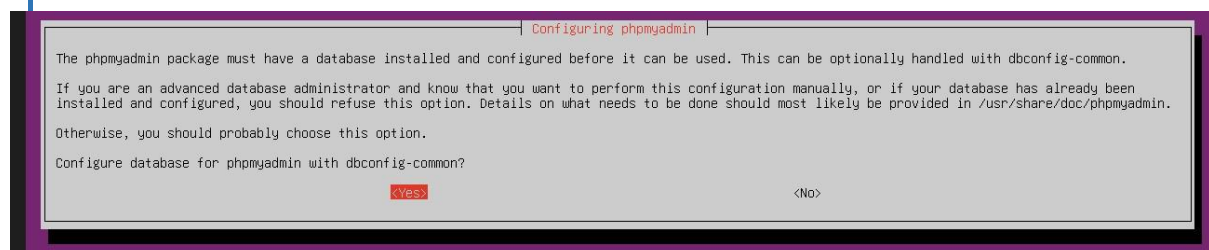
Install phpMyAdmin

```
$ sudo apt install phpmyadmin
... Done
```

Install phpMyAdmin for browser-based database management.



Select apache2 as the web server. Use Space to select, then Enter to confirm.



Choose Yes to let dbconfig-common handle the phpMyAdmin database setup automatically.



Enter a strong password for the phpMyAdmin MySQL user and confirm it.

Setup Database

Create the application database, user account, and import the schema.

Create database and import schema

```
MariaDB [(none)]> create database PIF_Db
->;
Query OK, 1 row affected (0.001 sec)
```

Create the application database named PIF_Db inside the MariaDB shell.

```
alfle266@alfle266:/var/www/html$ sudo mysql -u root -p PIF_Db < database.sql
Enter password:
```

Import the SQL schema file into PIF_Db. Enter the root password when prompted.

Install PHP extensions

```
alfle266@alfle266:/var/www/html$ sudo apt-get install php
Reading package lists... Done
Building dependency tree... Done
```

Install the base PHP package required by the application.

```
php_module (shared)
alfle266@alfle266:/var/www/html$ sudo apt install libapache2-mod-php
Reading package lists... Done
```

Install the Apache PHP module to enable PHP processing in the web server.

```
alfle266@alfle266:/var/www/html$ sudo apt install php-mysql
Reading package lists... Done
```

Install the PHP MySQL extension so the application can connect to MariaDB.

Create database user and grant privileges

```

MariaDB [(none)]> create user 'webuser'@'localhost' identified by '123';
Query OK, 0 rows affected (0.006 sec)

```

Create the application database user. Use a stronger password in production.

```

MariaDB [(none)]> grant all privileges on PIF_Db.* to 'webuser'@'localhost';
Query OK, 0 rows affected (0.006 sec)

MariaDB [(none)]> flush privileges
-> ;
Query OK, 0 rows affected (0.000 sec)

```

Re-grant privileges and flush to ensure all changes take effect immediately.

SSL Certificate Setup

Generate a self-signed SSL certificate and configure Apache to redirect all traffic to HTTPS.

Install OpenSSL and generate certificate

```

alfle266@alfle266:~$ sudo apt install openssl -y
[sudo] password for alfle266:

```

Install OpenSSL— required to generate the self-signed certificate and private key.

```

led, 0 to remove and 71 not upgraded.
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \-keyout /etc/ssl/private/apache.key \-out /etc/ssl/certs/apache.crt_

```

Generate a 2048bit RSA certificate valid for 365 days, saved to /etc/ssl/certs/apache.crt and /etc/ssl/private/apache.key.

```

-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:LU
State or Province Name (full name) [Some-State]:Luxemburg
Locality Name (eg, city) []:Luxemburg
Organization Name (eg, company) [Internet Widgits Pty Ltd]:EmileMetz
Organizational Unit Name (eg, section) []:1TPIFI
Common Name (e.g. server FQDN or YOUR name) []:AlfLe266
Email Address []:
alfle266@alfle266:~$

```

Fill in certificate details. Common Name should match your server's hostname or IP address.

Configure Apache for HTTPS

```
alfle266@alfle266:~$ sudo a2ensite default-ssl.conf
Enabling site default-ssl.
To activate the new configuration, you need to run:
systemctl reload apache2
```

Enable the default SSL virtual host. Run `systemctl reload apache2` as instructed in the output.

```
# SSLCertificateFile directive is needed.
SSLCertificateFile    /etc/ssl/certs/apache.crt
SSLCertificateKeyFile  /etc/ssl/private/apache.key_
# Server Certificate Chain:
```

Confirm the SSL config points to the certificate and key files generated in the previous step.

```
CustomLog ${APACHE_LOG_DIR}/access.log d
Redirect "/" "https://192.168.2.183/"
# For most configuration files from conf
```

Add a redirect rule in the HTTP virtual host to force all traffic to HTTPS.

```
alfle266@alfle266:~$ curl -I http://192.168.2.183
HTTP/1.1 302 Found
Date: Wed, 11 Mar 2026 19:07:26 GMT
Server: Apache/2.4.58 (Ubuntu)
Location: https://192.168.2.183/
Content-Type: text/html; charset=iso-8859-1
```

A 302 response with a `Location` header pointing to HTTPS confirms the redirect is working correctly.

Userstory L5

stationInput.php

Receiving data via POST method and inserting the measurements into the database.

```

1TPIFI > REPIF > stationInput.php
1  <?php
2  include_once("comCode.php");
3  NavBar("home");
4
5  $temperature = $_POST['temperature'];
6  $humidity    = $_POST['humidity'];
7  $pressure    = $_POST['pressure'];
8  $light       = $_POST['light'];
9  $gas         = $_POST['gas'];
10 $station     = $_POST['station_serial'];
11
12 $sqlInsert = $conn->prepare("INSERT INTO measurement (timestamp, temperature, humidity, airPressure, lightIntensity, airQuality, station) VALUES (?, ?, ?, ?, ?, ?, ?)");
13 $sqlInsert->bind_param("siiiiii", $_POST['timestamp'], $temperature, $humidity, $pressure, $light, $gas, $station);
14 $sqlInsert->execute();
15 ?>

```

Config.toml

Changes:

- Station_serial = 2
- Ip = "192.168.4.23"
- Path = "/1TPIFI/REPIF/stationInput.php"

alfle266@alfle266: /opt/station

```

GNU nano 8.4 config.toml
station_serial = 2          # Serial number of the station

interval = 2               # Time between measurements

toggle_intensity = 100     # The light level at which the LED should be turned on

[i2c]                     # I2C-Addresses of the sensors
bme280 = 0x76
bhl750 = 0x23
mql35 = 0x48

[destination]             # Information about the server
ip = "192.168.4.23"       # IP-Address or Hostname of the Server
path = "/1TPIFI/REPIF/stationInput.php" # Path to the script that accepts the measurements

```

Proof

I attached a video file of the database updating with new measurements as proof.



MicrosoftTeams-video(1).mp4

Userstory L5

1. Prerequisites

Before starting the backup server configuration, the following must be in place:

- Backup server VM is created and running Ubuntu Server 24.04 LTS
- Static IP address is configured (192.168.4.92)
- OpenSSH server is installed
- Web/database server is running MariaDB at 192.168.4.93
-

2. Install MySQL Client

Install the MySQL client software on the backup server to allow remote database connections:

```
g outdated hypervisor (qemu) binaries o
:~$ sudo apt install mysql-client -y
```

3. Create Backup Directory & User

Create the directory where backup files will be stored, and set up a dedicated system user to own it:

```
alfle266@alfle266backup:~$ sudo useradd -m backupuser
alfle266@alfle266backup:~$ sudo passwd backupuser
New password:
Retype new password:
passwd: password updated successfully
alfle266@alfle266backup:~$
```

4. Configure MariaDB on the Database Server

These steps are performed on the web/database server (192.168.4.93), not the backup server.

4.1 Create a MySQL Backup User

Log into MariaDB and create a user that can only connect from the backup server:

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> create user 'backupuser'@'192.168.4.92' identified by '123';
Query OK, 0 rows affected (0.007 sec)

MariaDB [(none)]> grant all privileges on pif_db.* to 'backupuser'@'192.168.4.92';
Query OK, 0 rows affected (0.017 sec)

MariaDB [(none)]> flush privileges;
Query OK, 0 rows affected (0.001 sec)
```

4.2 Allow External Connections

Edit the MariaDB configuration to allow connections from outside localhost:

```
alfle266@alfle266:/var/www/html$ sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Find the bind-address line and change it:

```
# Instead of skip-networking the default is now
# localhost which is more compatible and is not
bind-address            = 0.0.0.0
```

Then restart MariaDB:

```
sudo systemctl restart mariadb
```

5. Create the Backup Shell Script

Create the shell script on the backup server that performs the daily database backup:

```
sudo nano /usr/local/bin/db_backup.sh
```

Enter the following content:

```
GNU nano 7.2
#!/bin/bash

dbHost="192.168.4.93"
dbUser="backupuser"
dbName="PIF_Db"
backupDir="/backup/database"
date=$(date +%Y-%m-%d_%H-%M-%S)
backupFile="$backupDir/${dbName}_${date}.sql"

mysqldump -h $dbHost -u $dbUser PIF_Db "$backupFile"
```

Save and close the file, then assign execute permissions:

```
alfle266@alfle266backup:~$ sudo chmod +x /usr/local/bin/db_backup.sh
[sudo] password for alfle266:
alfle266@alfle266backup:~$
```

6. Schedule Daily Backup at 8:00 AM

Configure the cron job to run the backup script automatically every day at 8:00 AM:

```
crontab -e
```

Add the following line at the bottom of the crontab file:

```
# m h dom mon dow   command
0 8 * * * /usr/local/bin/db_backup.sh
```

This will execute the script every day at exactly 08:00.

7. Configure FTP Access

Install the vsftpd FTP server and configure access to the backup directory:

7.1 Install vsftpd

```
TX errors 0 dropped 0 overruns 0 carrier 0 coll
alfle266@alfle266backup:~$ sudo apt install vsftpd -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

7.2 Create FTP User

```
alfle266@alfle266backup:~$ sudo passwd ftpuser
New password:
Retype new password:
passwd: password updated successfully
alfle266@alfle266backup:~$
passwd: password updated successfully
alfle266@alfle266backup:~$ sudo chown ftpuser:ftpuser /backup/database
alfle266@alfle266backup:~$
```

7.3 Configure vsftpd

Edit the vsftpd configuration file:

```
sudo nano /etc/vsftpd.conf
```

Ensure the following lines are set (uncomment if necessary):

```
#
# Uncomment this to allow local users to log in.
local_enable=YES
write_enable=YES
#
```

Restart and enable the FTP service:

```
sudo systemctl restart vsftpd
sudo systemctl enable vsftpd
```

8. Verify the Backup

To manually run the script and verify a backup is created:

```
sudo sh /usr/local/bin/db_backup.sh  
ls -lh /backup/database/
```

```
Warning: column statistics not supported by the server.  
alfle266@alfle266backup:/backup/database$ ls  
pif_db_2026-04-16_13-24-42.sql  pif_db_2026-04-16_13-25-18.sql  pif_db_2026-04-16_13-30-33.sql  
alfle266@alfle266backup:/backup/database$
```